

◆ What is Asynchronous Programming in C#?

Asynchronous programming is a programming paradigm that allows your application to perform **non-blocking operations**, meaning a task can run in the background while the main thread continues executing.

In simple terms:

Instead of waiting for a task to complete, your program continues doing other work and gets notified when the task is done.

◆ Why Async Programming?

✗ Problem with Synchronous Code

In synchronous execution:

- Each task waits for the previous one to complete
- UI freezes (in desktop/web apps)
- Poor performance for I/O operations

```
public string GetData()
{
    Thread.Sleep(5000); // Simulates delay
    return "Data Loaded";
}
```

👉 This blocks the thread for 5 seconds.

✅ Async Solution

- Improves responsiveness (especially in ASP.NET, UI apps)
 - Better scalability (handles more requests)
 - Efficient use of threads
-

◆ Key Concepts in Async Programming

1. async Keyword

- Marks a method as asynchronous
- Allows usage of await

2. await Keyword

- Waits for a task to complete **without blocking the thread**

3. Task / Task<T>

- Represents an asynchronous operation
 - Task<T> returns a value
-

◆ Basic Example

✓ Synchronous Version

```
public string FetchData()
{
    Thread.Sleep(3000);
    return "Data Received";
}
```

✓ Asynchronous Version

```
public async Task<string> FetchDataAsync()
{
    await Task.Delay(3000); // Non-blocking delay
    return "Data Received";
}
```

◆ How It Works Internally

- async method starts execution
- When it hits await, it:
 - Returns control to caller
 - Registers continuation
- Once task completes → execution resumes

👉 No thread is blocked during waiting.

◆ Real-Time Example (API Call Simulation)

```
public async Task<string> GetApiDataAsync()
{
    using (HttpClient client = new HttpClient())
    {
        var response = await client.GetAsync("https://api.example.com/data");
        return await response.Content.ReadAsStringAsync();
    }
}
```

👉 Used heavily in:

- ASP.NET Core Web APIs
 - Microservices
 - External API integrations
-

◆ Calling Async Method

```
public async Task Main()
{
    string result = await FetchDataAsync();
    Console.WriteLine(result);
}
```

```
}
```

◆ Important Rules

✓ Rule 1: Use async with await

```
public async Task DoWorkAsync()
```

✓ Rule 2: Avoid async void (Except Event Handlers)

```
// ❌ Bad
```

```
public async void SaveData()
```

```
// ✅ Good
```

```
public async Task SaveDataAsync()
```

✓ Rule 4: Naming Convention

- Always suffix with **Async**

```
GetDataAsync()
```

```
SaveChangesAsync()
```

◆ Types of Tasks

Type	Description
Task	No return value
Task<T>	Returns value
ValueTask<T>	Optimized for performance (advanced)

◆ Parallel Execution Example

```
public async Task RunTasksAsync()
{
    Task task1 = Task.Delay(2000);
    Task task2 = Task.Delay(3000);

    await Task.WhenAll(task1, task2);

    Console.WriteLine("Both tasks completed");
}
```

👉 Runs tasks **in parallel**, not sequentially.

◆ Async in ASP.NET Core

```
public async Task<IActionResult> GetData()
{
    var data = await _service.GetDataAsync();
    return Ok(data);
}
```

👉 Improves:

- Throughput
 - Scalability
 - Thread utilization
-

◆ When to Use Async?

✓ **Use Async for:**

- Database calls
- API calls
- File I/O operations
- Network operations

✗ **Avoid Async for:**

- Simple computations
- Small in-memory operations

◆ **Summary**

- Async programming = **non-blocking execution**
 - Uses async, await, and Task
 - Improves performance and scalability
 - Essential for modern C# applications (especially ASP.NET Core)
-